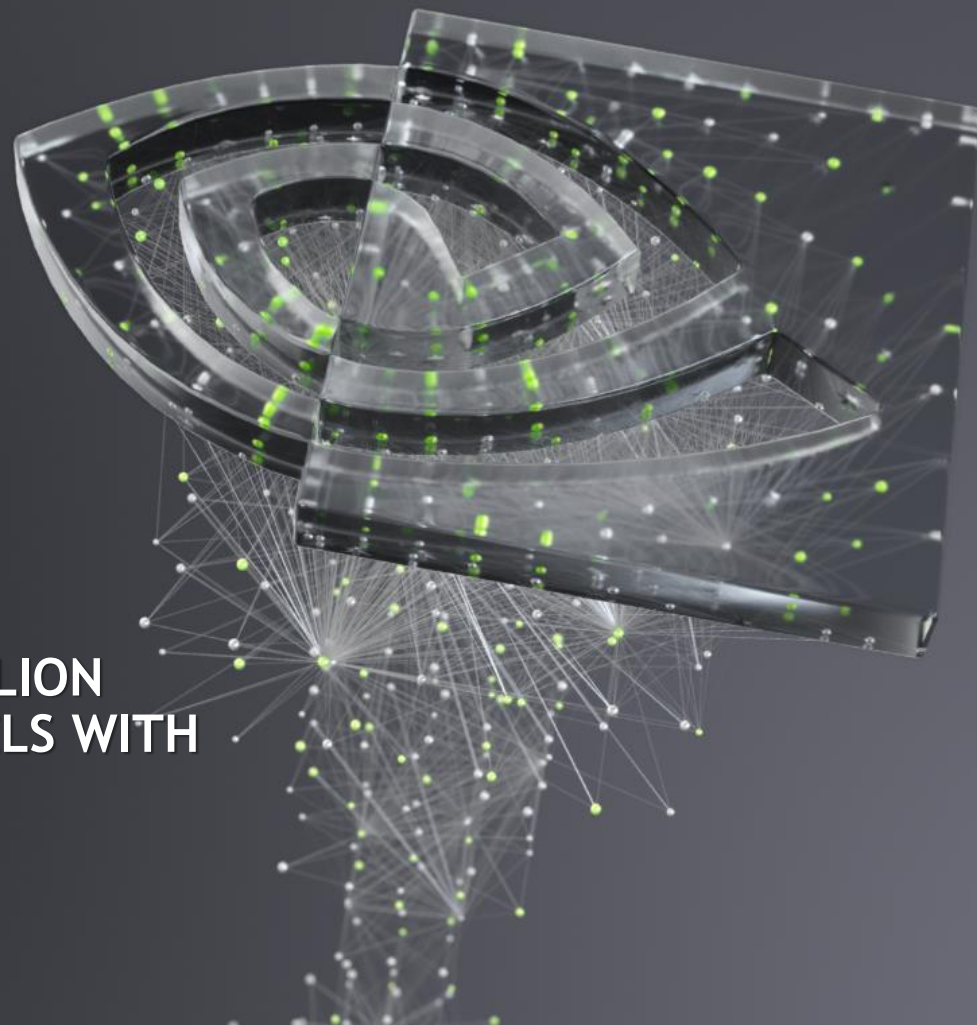




MEGATRON-LM: TRAINING BILLION PARAMETER LANGUAGE MODELS WITH GPU MODEL PARALLELISM

Raul Puri, 03/06/2020



WHAT IS MEGATRON?

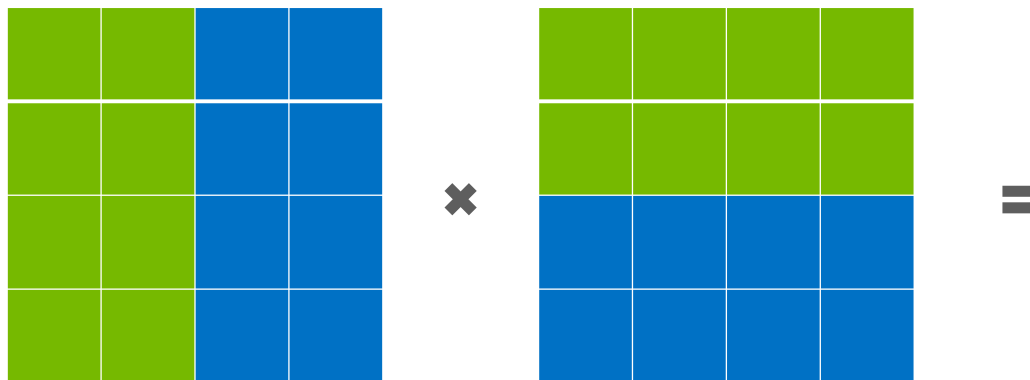
- ▶ Paper: <https://arxiv.org/abs/1909.08053>
- ▶ Repo: <https://github.com/NVIDIA/Megatron-LM>

NVIDIA's framework for efficiently training the world's largest language models



MODEL PARALLELISM

Parallel GEMMs



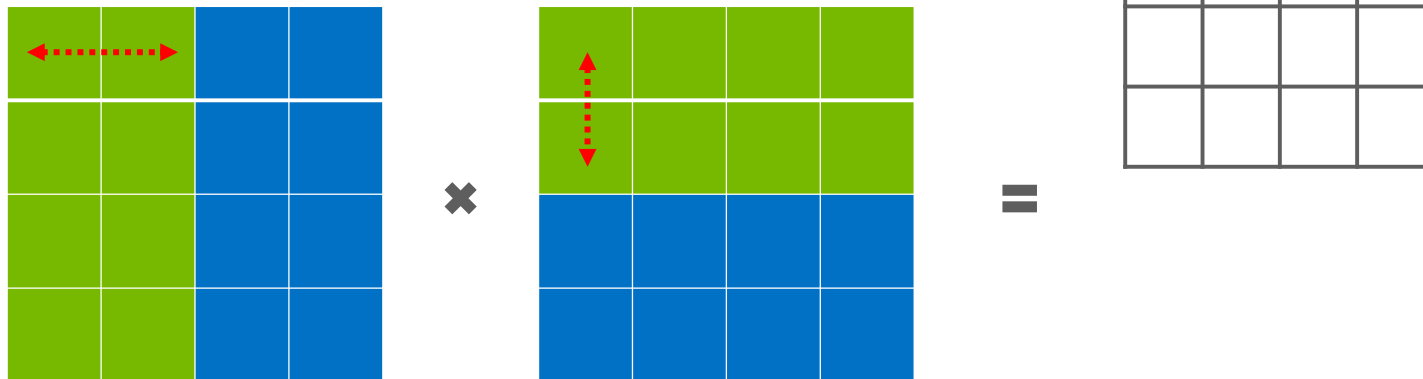
$$X = [X_1, X_2]$$

$$A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$$

$$Y = Y_1 + Y_2$$

MODEL PARALLELISM

Parallel GEMMs



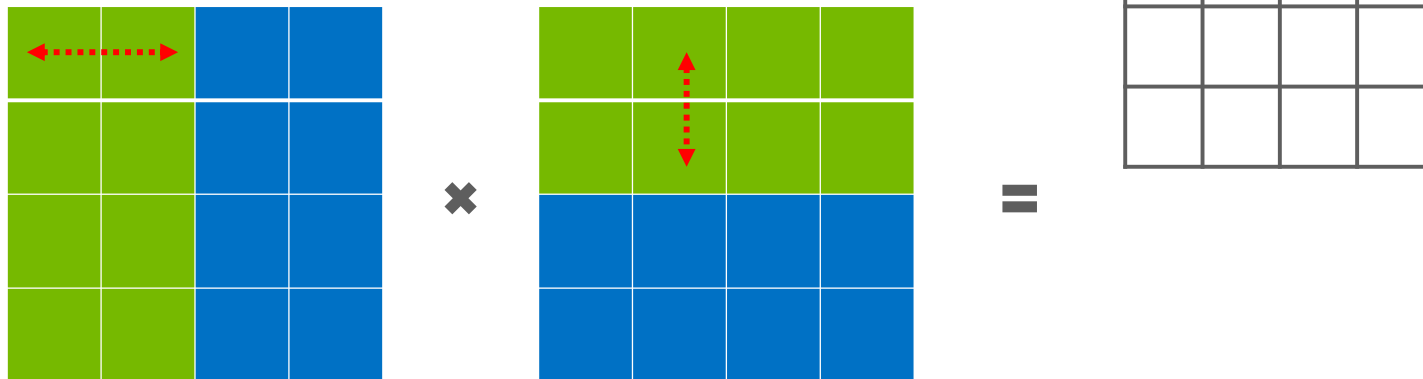
$$X = [X_1, X_2]$$

$$A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$$

$$Y = Y_1 + Y_2$$

MODEL PARALLELISM

Parallel GEMMs



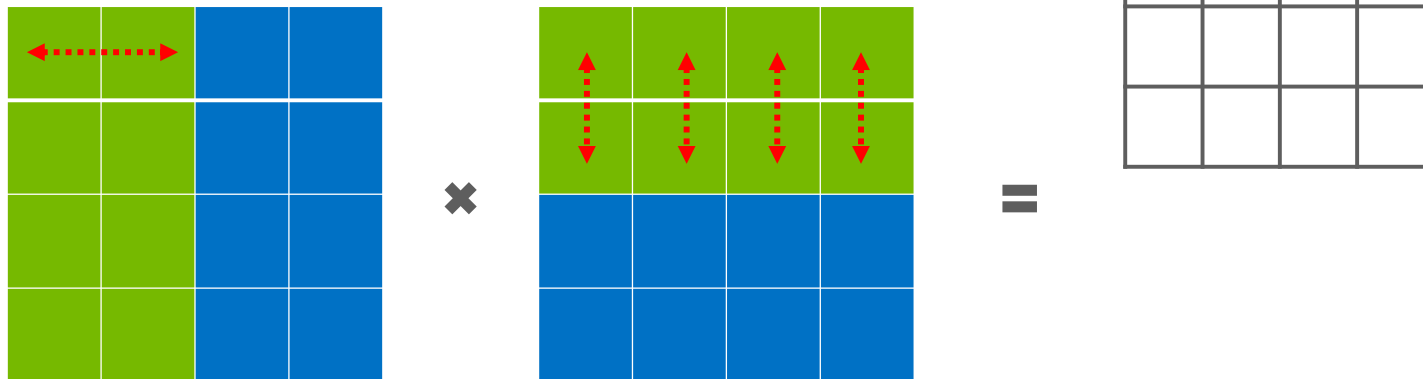
$$X = [X_1, X_2]$$

$$A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$$

$$Y = Y_1 + Y_2$$

MODEL PARALLELISM

Parallel GEMMs



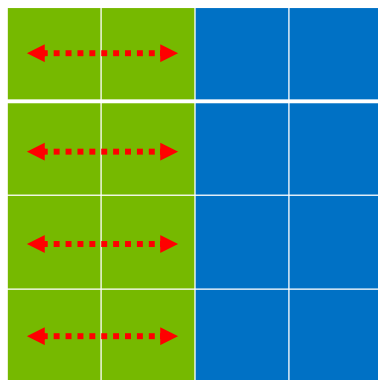
$$X = [X_1, X_2]$$

$$A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$$

$$Y = Y_1 + Y_2$$

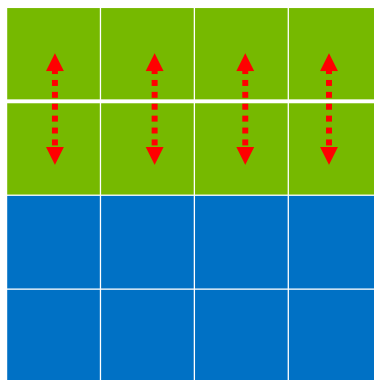
MODEL PARALLELISM

Parallel GEMMs

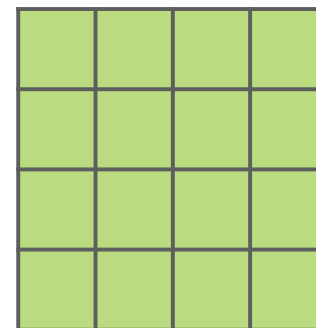


$$X = [X_1, X_2]$$

×



=

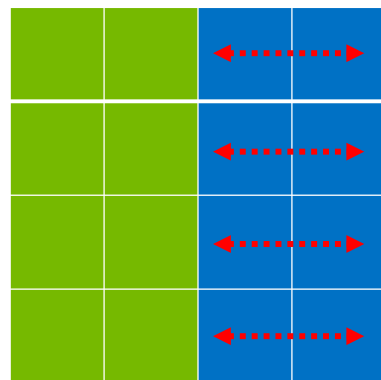


Y_1

$$Y = Y_1 + Y_2$$

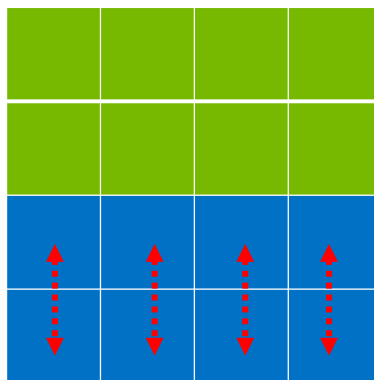
MODEL PARALLELISM

Parallel GEMMs



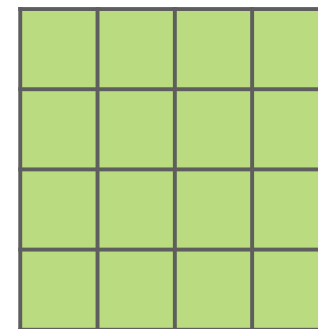
$$X = [X_1, X_2]$$

×

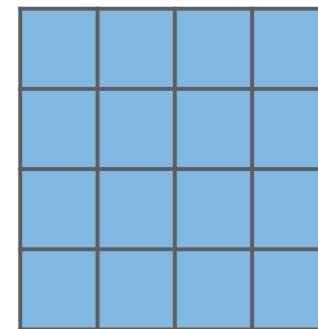


$$A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$$

=



+



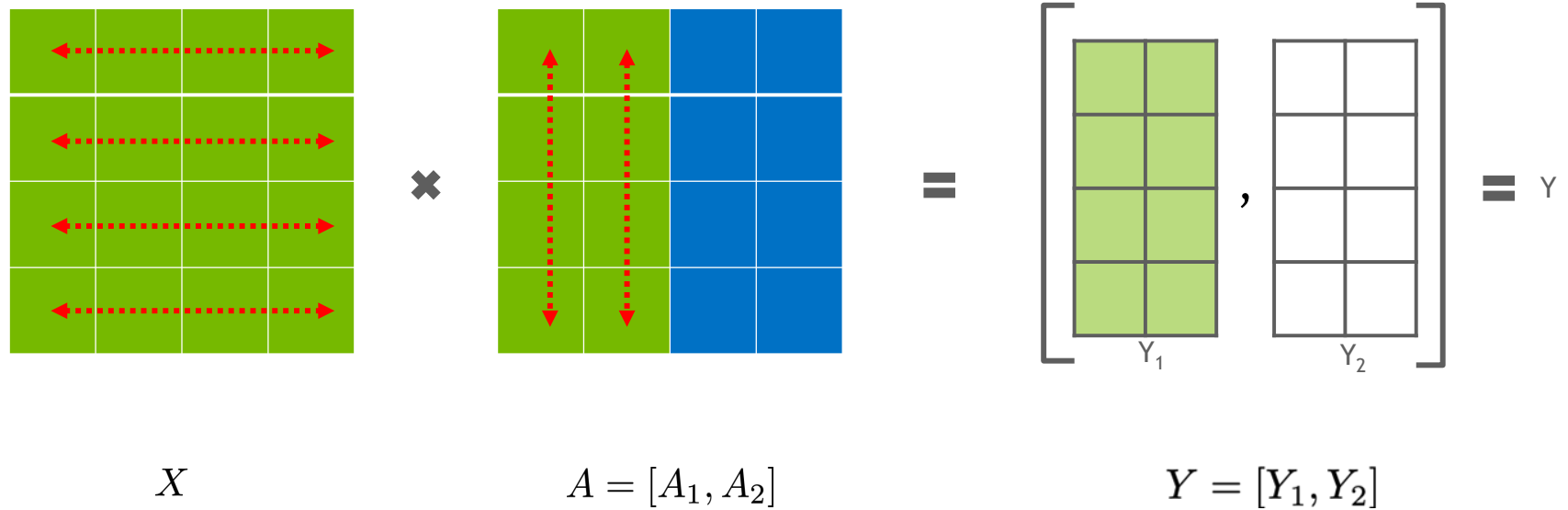
=

Y

$$Y = Y_1 + Y_2$$

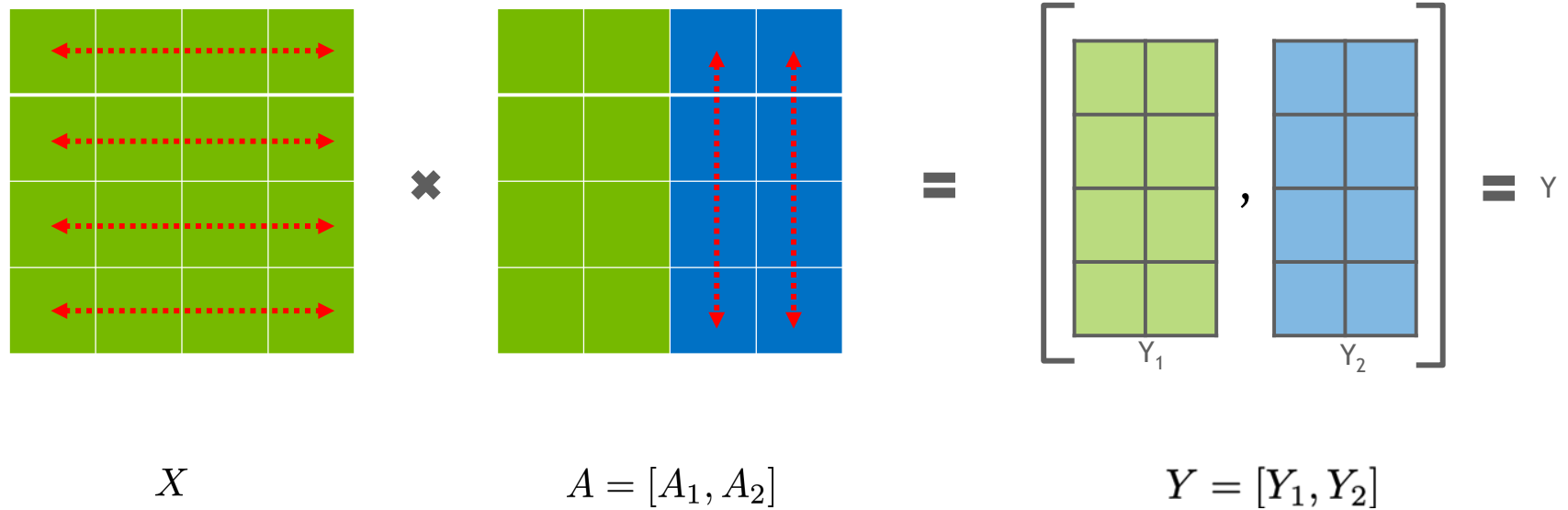
MODEL PARALLELISM

Parallel GEMMs



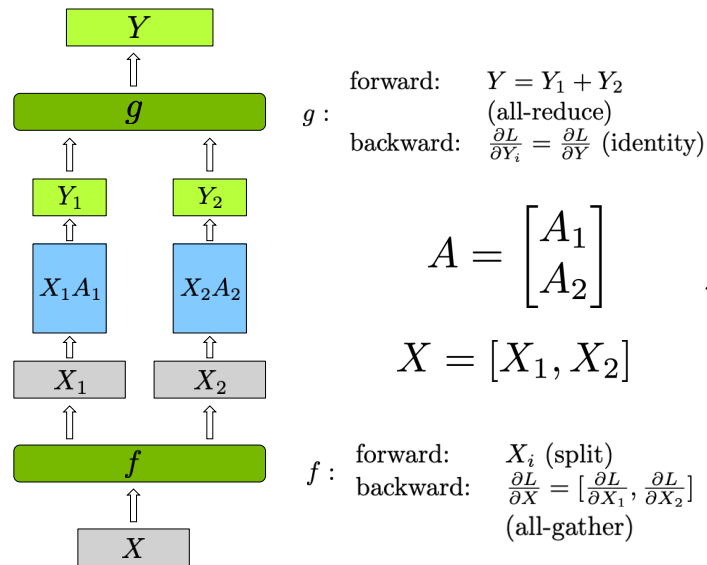
MODEL PARALLELISM

Parallel GEMMs

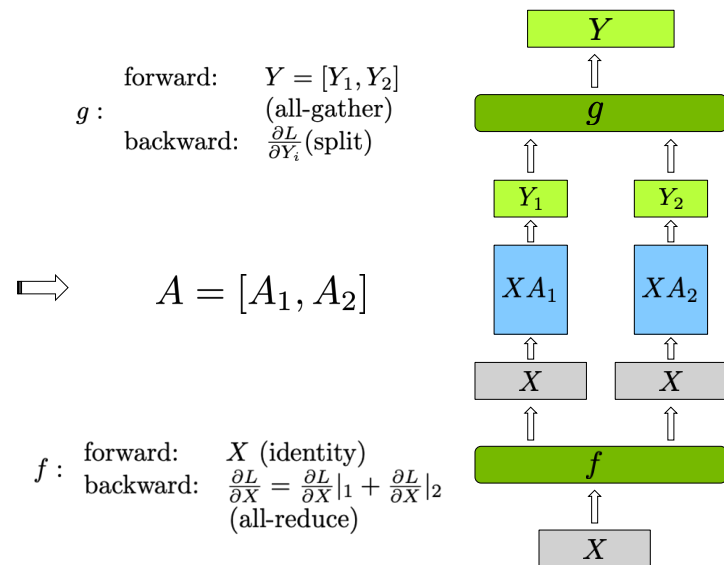


MODEL PARALLELISM

Row Parallel Linear Layer



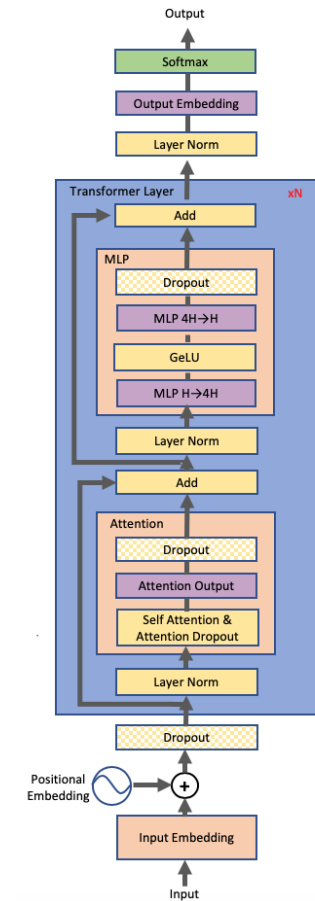
Column Parallel Linear Layer



APPROACH

Transformer Goals

- ▶ Group math heavy operations (such as GEMMs) to minimize parallel sync points
- ▶ Develop an approach that can be fully implemented with the insertion of a few simple collectives
 - ▶ Rely on pre-existing NCCL/PyTorch operations for a native PyTorch implementation
- ▶ Use Volta's tensor cores for mixed precision training



APPROACH

MLP

- MLP:

$$Y = \text{GeLU}(XA)$$

$$Z = \text{Dropout}(YB)$$

- Approach 1: split X column-wise and A row-wise

$$X = [X_1, X_2] \quad A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix} \longrightarrow Y = \text{GeLU}(X_1 A_1 + X_2 A_2)$$

- Requires synchronization before GeLU

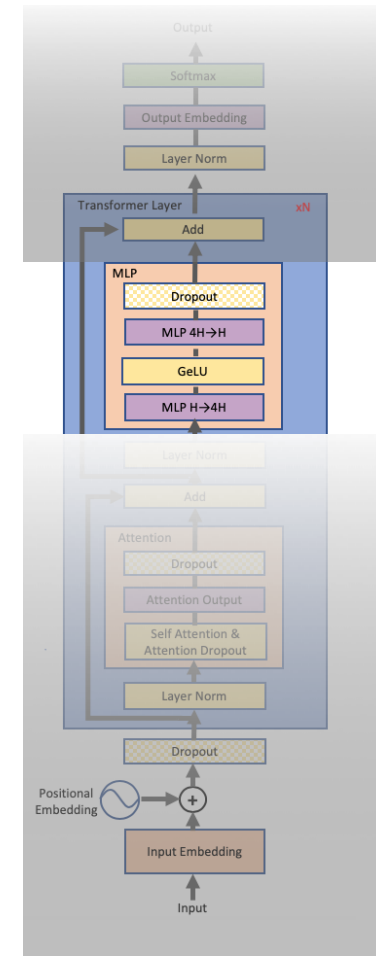
GeLU of sums != sum of GeLUs

- Approach 2: split A column-wise

$$A = [A_1, A_2] \longrightarrow [Y_1, Y_2] = [\text{GeLU}(X A_1), \text{GeLU}(X A_2)]$$

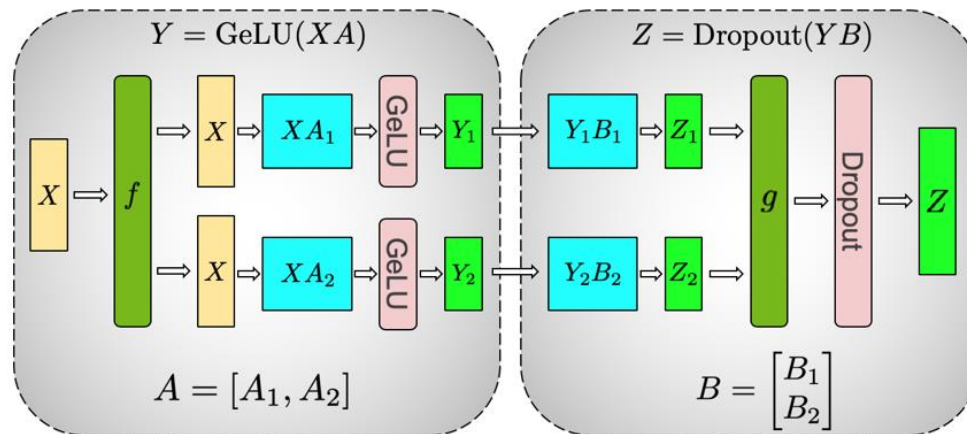
- No synchronization necessary

- Gather/all-reduce rely on pre-existing NCCL/PyTorch operations for a native PyTorch implementation 🍏

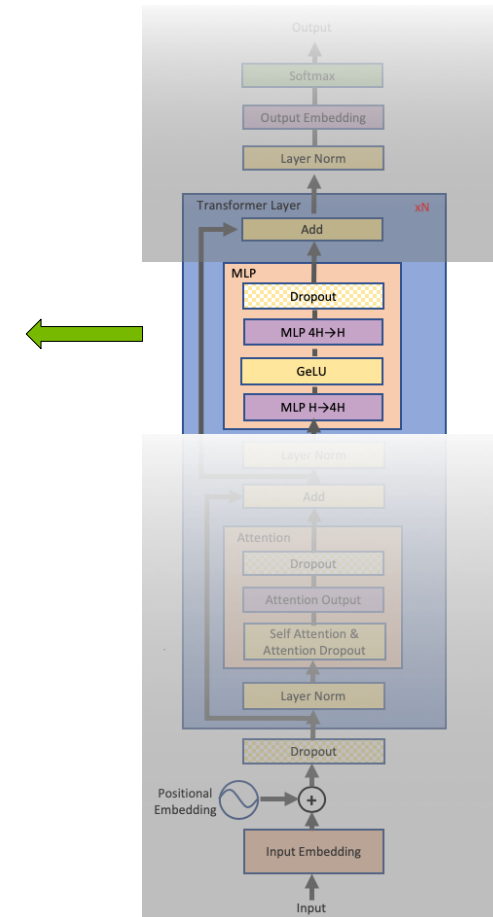


APPROACH

Fused MLP



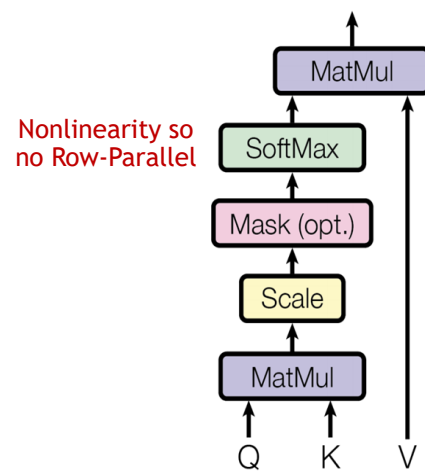
f and g are **conjugate**, f is **identity** operator in the forward pass and **all-reduce** in the backward pass while g is **all-reduce** in forward and **identity** in backward.



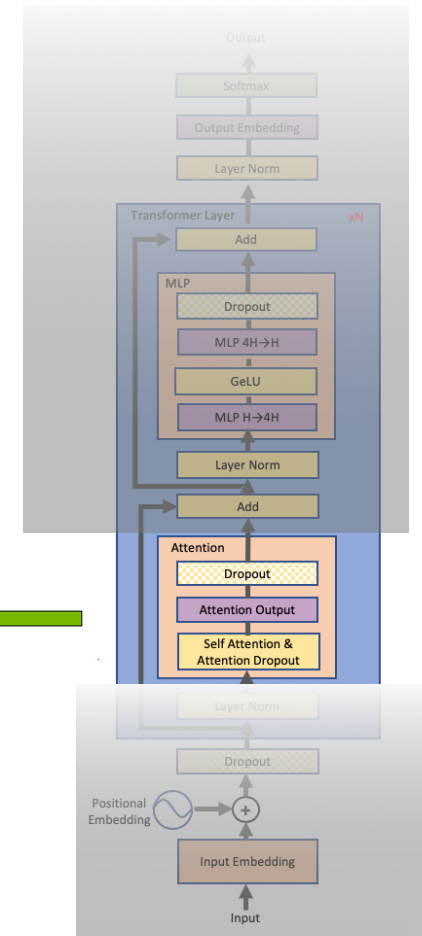
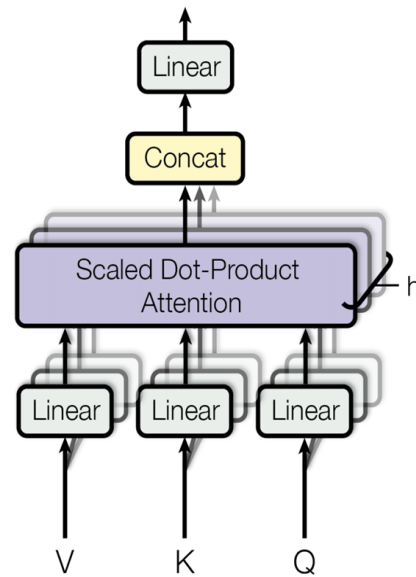
APPROACH

Fused Self-Attention

Scaled Dot-Product Attention

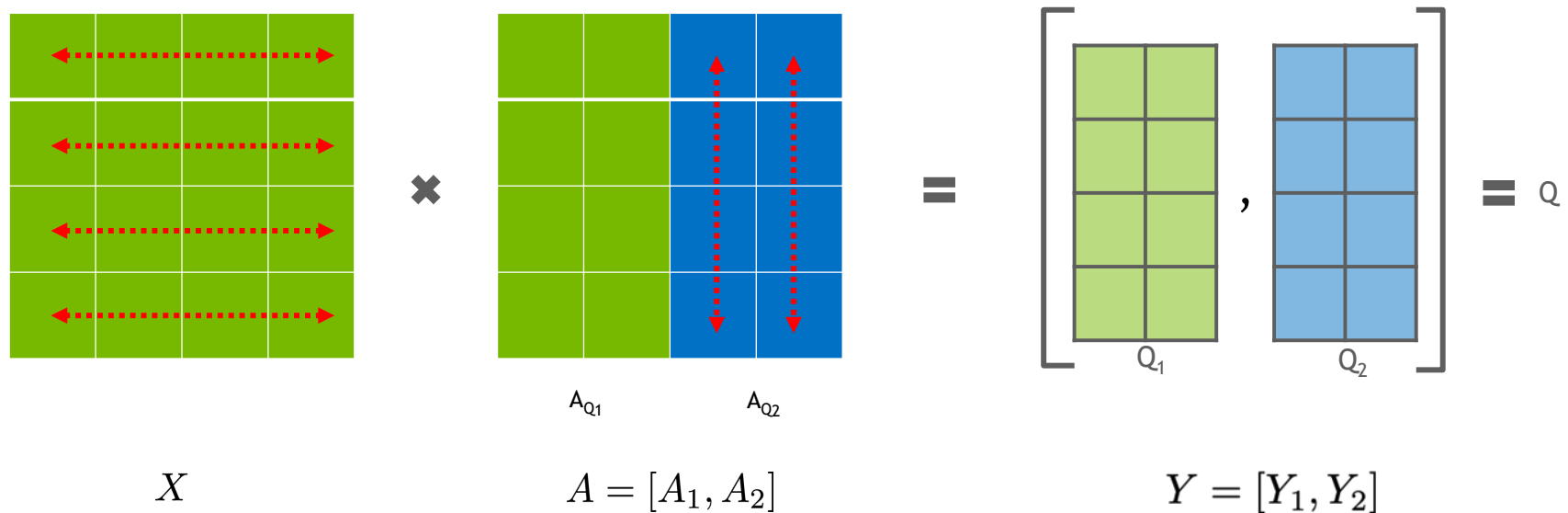


Multi-Head Attention



APPROACH

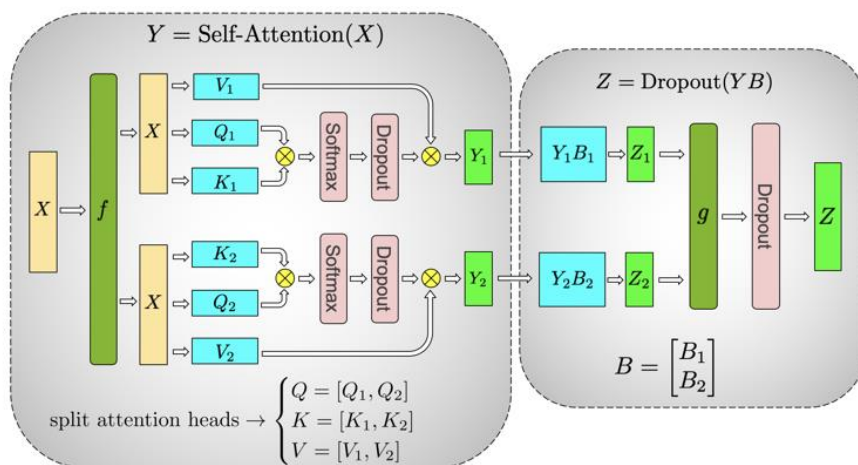
Fused Self-Attention



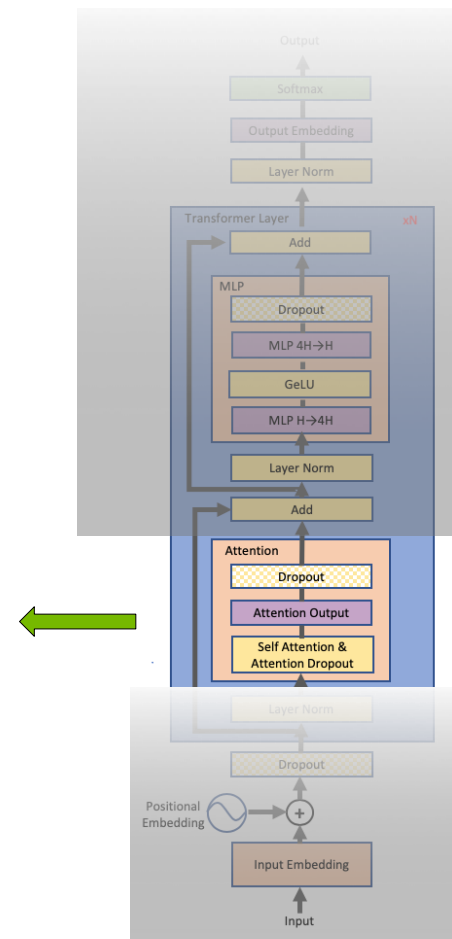
Attention heads can be parallelized with Column Parallel GEMMs (ex. Query head 1 (Q_1) and Query head 2 (Q_2))

APPROACH

Fused Self-Attention

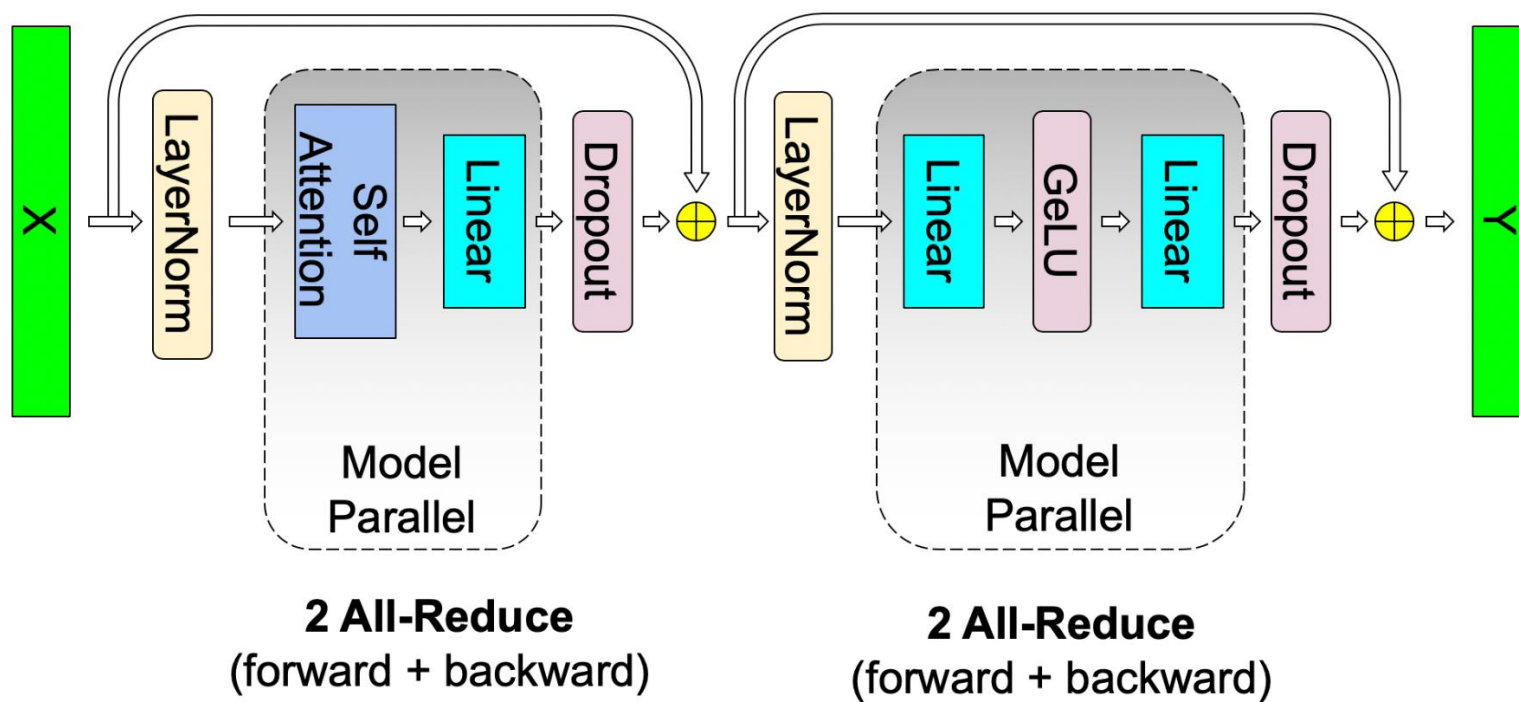


f and g are **conjugate**, f is **identity** operator in the forward pass and **all-reduce** in the backward pass while g is **all-reduce** in forward and **identity** in backward.



APPROACH

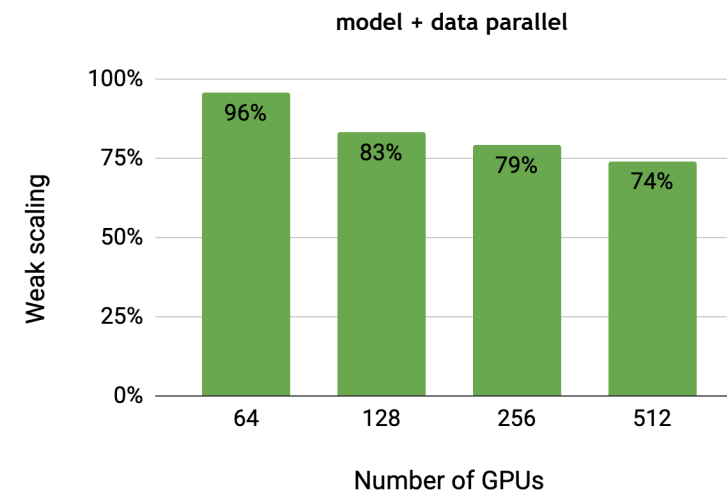
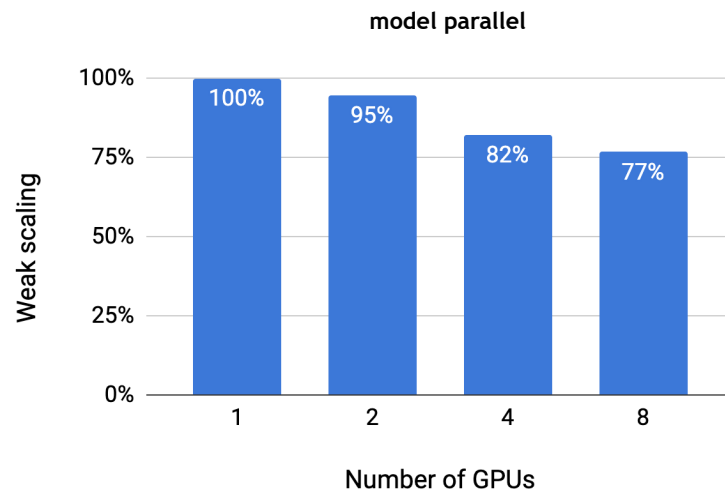
Putting It All Together: Parallel Transformer Layer



APPROACH

Weak Scaling

Config	Hidden size	Attention heads	Number of layers	Number of parameters (billions)	Model parallel GPUs	Model+data parallel GPUs
1	1536	16	40	1.2	1	64
2	1920	20	54	2.5	2	128
3	2304	24	64	4.2	4	256
4	3072	32	72	8.3	8	512



Baseline (1.2B parameters on a single GPU) achieves **39 TeraFLOPs per second**, i.e. **30% of the theoretical peak** during the entire training process

